

Conversa — Testing guide

A click-by-click walkthrough to exercise every feature and see what works.

For how it's built, see ARCHITECTURE.md.

0. Start the app

```
npm install      # first time only
npm run dev      # or: node node_modules/next/dist/bin/next dev (if npm not on PATH)
```

Open **http://localhost:3000**.

> The database is already migrated and seeded. To reset demo data at any time:

> `npm run db:seed` (this clears and recreates the demo org).

Demo accounts (all share the password `ConversaDemo!23`)

Email	Role	What they can see
ava.owner@conversa.demo	Owner	Everything
noah.manager@conversa.demo	Support Manager	Inbox, contacts, AI, chatbots, rep...
priya.agent@conversa.demo	Support Agent	Inbox, contacts, knowledge (no bro...
sara.marketing@conversa.demo	Marketing	Broadcasts, contacts, reports

1. Marketing site (no login)

- Go to `/`. **Expect:** announcement bar, hero “Every conversation. One intelligent workspace.”, a four-column inbox preview, feature sections, pricing (marked as demo), footer.
 - Top-nav links (AI Agents, Pricing, About...) and **Start free / Sign in**.
-

2. Authentication

- **Register:** `/register` → enter a name, email, password (≥8 chars, a letter + a number). **Expect:** if email confirmation is on in Supabase, a “check your inbox” message; otherwise you go to `/onboarding`.
- **Protected routes:** while signed out, visit `/app/inbox`. **Expect:** redirect to `/login`.
- **Login:** `/login` with `ava.owner@conversa.demo` / `ConversaDemo!23`. **Expect:** redirect to `/app/inbox`.

- **Forgot/reset:** </forgot-password> → enter an email → **Expect:** a neutral “if that email exists...” confirmation (no account enumeration).
 - **Sign out:** top-right avatar → **Sign out** → back to </login>.
-

3. Onboarding (create a new organisation)

- Sign out, register a brand-new email, or use the org switcher →

New organisation.

- At </onboarding>, enter an organisation name + industry → **Create workspace**.

Expect: a new org with a default team + inbox is created, you become its

Owner, and you land on an (empty) inbox. This proves tenant creation and

isolation — the new org sees none of Grovefield's data.

4. Roles & organisation switching

- As Owner, click the **org selector** (top bar). **Expect:** your orgs listed; switching sets the active org.
 - Sign in as [priya.agent@conversa.demo](#). **Expect:** the left icon sidebar shows **fewer** modules (no Broadcasts/Integrations/Settings) — role gating.
 - Sign in as [sara.marketing@conversa.demo](#). **Expect:** Broadcasts, Contacts, Reports emphasised; no Inbox management tools.
-

5. Shared inbox (core)

Sign in as **Owner** and open **Inbox**.

- **Views:** click My Inbox / Unassigned / Waiting / Resolved / AI Handled / Spam. **Expect:** the list and the counts change per view.
- **Open a conversation:** click a row. **Expect:** column 3 shows the message timeline (customer + agent/AI bubbles, internal notes, AI summary), column 4 shows Ticket + Contact + Sentiment.
- **Reply:** type in the composer → **Send**. **Expect:** your message appears at the bottom, the list preview + timestamp update.
- **Internal note:** switch to **Internal note**, type, **Add note**.
Expect: a yellow internal-note block (not sent to the customer).
- **AI draft (demo):** click **AI draft**. **Expect:** a labelled draft is inserted into the composer for you to review — clearly demo mode.
- **Send & resolve:** type a reply → **Send & resolve**. **Expect:** message

sent and the conversation becomes Resolved.

- **Assign / priority / resolve:** use the header buttons — **Assign to me**, the **priority** menu, **Resolve/Reopen**. **Expect:** toasts confirm and the details panel updates.

5b. Realtime (live updates) — the fun one

- Keep the **Inbox** open in your browser. Note the green • **Live** indicator next to the list header.

- In a terminal, emit an inbound message from “a customer”:

```
npx tsx scripts/emit-test-message.ts "Any update on my order? (realtime test)"
```

- **Expect (no reload):** the target conversation jumps to the top of the list with the new preview and a fresh timestamp; if it's the open thread, the new inbound bubble appears in the timeline within ~1s.

6. Contacts

Open **Contacts**. **Expect:** a table of seeded contacts with name, email, phone, company and consent badge.

6b. CSV import

- **Import CSV** → the import wizard.
- **Upload** a **.csv** whose first row is headers (e.g. `First Name,Last Name,Email,Phone,Company,Tags`). Drag-drop or pick a file (≤5 MB).
- **Map & review:** columns are auto-mapped from the headers — adjust any.

Expect: validation counts (ready to import / invalid email / no email or phone / duplicates in file) and a live preview of the first rows.

- **Consent:** choose ***Unknown*** or ***Opted in*** (+ a source) — consent is recorded per contact and gates who can receive broadcasts.
- **Import. Expect:** a summary — ****Created / Updated / Skipped / Tags linked. Existing contacts (matched by email or phone) are updated**, not duplicated; rows with no email/phone are skipped; invalid emails are dropped but the contact is still imported if it has a phone. New tags are created and linked. Open **Contacts** to see the results.**

7. Reports

Open **Reports**. **Expect:** KPI cards (total, resolved, AI containment %, SLA

breaches), a **Conversations by channel** bar chart and a **By status** pie chart, plus response-time tiles. Numbers are derived from the seeded conversations.

8. AI Agents

Open **AI Agents**. **Expect:** a **demo-mode** banner and the seeded “Grove Assistant” card (published, friendly tone, channels). Configure / sandbox buttons are placeholders for the agent build-out.

9. Knowledge base

Open **Knowledge**. **Expect:** 10 seeded articles with source type, index status (indexed) and published/draft badges.

10. Chatbot builder

Open **Chatbots**.

- **Open the seeded flow:** “Website Welcome Bot” → **Open builder**.
 - **Canvas:** drag the background to **pan**; use the bottom-left **zoom** controls; see the **minimap** (bottom-right) and the ● **Valid** status.
 - **Add a node:** click any node in the left **palette** (e.g. *Ask Question*).
- Expect:** it appears on the canvas and is selected; edit its fields in the right **settings panel**.
- **Connect:** click a node's **output handle** (small dot on its bottom edge), then click a **target node**. **Expect:** a cyan arrow connects them. Click an edge to delete it.
 - **Move nodes:** drag a node — it repositions and edges follow.
 - **Validate:** delete the connection into a node → the toolbar shows an **issue** count; open it to see “... has no outgoing connection” / “unreachable”.
 - **Test simulator:** click **Test** → **Start test**. Play the flow: you'll see the welcome message and choice buttons; pick **Talk to a human** → **Expect:** it routes to the **Human Handoff** terminal step.
 - **Save / Publish:** **Save** stores the draft; **Publish** validates, creates a new **version** (see the **v#** history) and marks the flow published.
- Duplicate** clones the flow.
- **New flow:** back on the list, **New flow** creates a blank flow (just a Start node) and opens the builder.
-

10b. Automations

Open **Automations** (sign in as Owner or Manager).

- **List:** see the 3 seeded rules with an order number, trigger, condition/ action counts and an **Active/Off** toggle.
- **Reorder:** drag a rule by its grip handle to change its order — the new order is saved.
- **Enable/disable:** click a rule's toggle. **Expect:** a toast and the badge flips between Active/Off.
- **Edit:** click the pencil to open the **When** → **If** → **Then** editor. Change the trigger, **Add condition** (field / operator / value), **Add action** (type + parameter), then **Save**. **Expect:** a “Rule saved” toast.
- **Test:** click **Test**. **Expect:** the right panel shows whether the conditions **matched** your most recent conversation, a / per condition, which actions **would** run, and a new entry under **Recent runs**.
- **New rule:** back on the list, **New rule** creates a blank disabled rule and opens its editor.

Rules fire live on inbound messages

The seeded rule **“Route billing keywords to Billing team”** runs automatically:

```
npm run wa:sim "Please process a refund for order 5501"
```

Expect: the WhatsApp conversation is auto-assigned to the **Billing** team and tagged **Billing** (open it in the Inbox to see), and an entry appears under the rule's **Recent runs**. The engine fires rules in order with loop protection (each rule fires once per cascade; follow-on events recurse only to a max depth).

Time-based rules (scheduled sweep)

Triggers that need a clock — `conversation.waiting`, `conversation.idle`, `sla.at_risk` — run via the sweep, which also marks overdue conversations `sla_breached`:

```
npm run sweep # run once locally
# or hit the endpoint a scheduler would call:
curl "http://localhost:3000/api/cron/sweep?secret=$CRON_SECRET"
```

Expect: a JSON summary `{orgs, slaBreached, fired, candidates}`. Create an active rule with a `conversation.waiting` trigger (e.g. 30 min) to see it act on idle conversations; the sweep fires each rule **once per conversation** and a wrong/missing `CRON_SECRET` returns **401**. Wire the endpoint to a scheduler

(Vercel Cron / Supabase / GitHub Actions) in production.

11. Broadcasts (composer)

Open **Broadcasts** (Owner or Marketing).

- **New broadcast** → opens the composer (a new draft).
 - **Message:** choose a channel, optionally pick the approved `spring_sale` template, or write a body. Click the `{{first_name}}` chip to insert a variable — the **Preview** updates for a sample contact.
 - **Audience:** click **Preview audience**. **Expect:** with WhatsApp + “Require consent”, roughly **8 eligible / 22 no-consent / 0 suppressed / 30 total** — consent is enforced (only opted-in contacts qualify). Add tag filters to narrow it.
 - **Schedule:** choose Send now or Schedule (+ optional frequency cap).
 - **Approve** → **Send now**. **Expect:** the status becomes **sent** and the right rail shows a **Delivery** breakdown (queued/sent/delivered/read/replied/failed) across the eligible recipients.
- *(Demo mode — recipients are recorded and delivery is simulated; no live provider is contacted.)*
- **Send a test:** enter an address and click the send icon → a demo test is logged.

11b. Live WhatsApp webhook

Test the official Meta Cloud API inbound webhook locally (no real number needed):

- Ensure `.env.local` has `WHATSAPP_APP_SECRET`, `WHATSAPP_VERIFY_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, and that the dev server was (re)started after setting them.
- Verification handshake:
`GET /api/webhooks/whatsapp?hub.mode=subscribe&hub.verify_token=<token>&hub.challenge=42`
→ returns `42`.
- Inbound message + idempotency:

```
npm run wa:sim
```

Expect: `{messages:1}` then a second `{duplicates:1}` (same id ignored).

Open **Inbox** → a **Test Customer** WhatsApp conversation appears (live, via realtime).

- Delivery status:

```
npm run wa:sim status
```

Expect: a message then `{statuses:1}` (marked read).

- A tampered/missing signature returns **401** (verified in the unit tests).
- **Outbound send:** open the WhatsApp conversation in the **Inbox** and send a reply. **Expect:** the message posts and is stored with a provider id (`external_id`). Without `WHATSAPP_ACCESS_TOKEN` it's a labelled demo send (`metadata.demo = true`); with a real token it dispatches via the Graph API and later delivery webhooks update its status.

11c. Live email adapter (Resend)

- Ensure `.env.local` has `RESEND_WEBHOOK_SECRET` (and optionally `RESEND_API_KEY` + `EMAIL_FROM`); restart the dev server after changes.
- Inbound + idempotency:

```
npm run email:sim
```

Expect: `{inbound:1}` then `{duplicates:1}`. Open **Inbox** → a new email conversation from **new.lead@example.com** ("Question about bulk orders").

- Outbound: open that email conversation and send a reply. **Expect:** it's stored with an `external_id` (`email_DEMO_...` in demo mode; a real Resend id with `RESEND_API_KEY` set, after which delivery events update its status).
- A bad Svix signature returns **401** (verified in the unit tests).

12b. Integrations / Settings / Notifications

- **Integrations:** all six channels shown; seeded ones display **Demo mode**. Live connection forms are the adapter build-out.
- **Settings:** a full area with a sub-nav (owner/admin can manage; others view):
 - **General** — organisation details. **My Profile** — your account + role.
 - **Team & Roles** — change a member's role or remove them (not yourself).
 - **Inboxes** — add/delete inboxes, set the default.
 - **Tags** — create (with colour) / delete tags.
 - **SLA Policies** — create/edit/delete first-response & resolution targets.
 - **Business Hours** — per-day open/close + timezone.
 - **Billing** — switch between demo plans (no payment taken).
 - **Notifications:** empty-state until mentions/assignments/SLA events are wired.

12. Verify multi-tenant isolation (security)

- Note a contact/conversation in Grovefield (as Owner).
- Create a **new organisation** via onboarding (a fresh workspace).
- Switch to it. **Expect:** the inbox, contacts and reports are **empty** — none

of Grovefield's data leaks across. This is enforced by RLS, not just the UI.

13. Quality gates & automated tests

```
npm run typecheck      # expect: no errors
npm run lint           # expect: no errors
npm run build          # expect: production build succeeds (all routes compile)
npm run test:unit      # Vitest unit tests (no DB) – expect all pass
npm run test:integration # Vitest RLS/tenant-isolation + ticket transitions (needs .env.local)
npm test               # unit + integration together
npm run test:e2e       # Playwright auth + inbox (needs a running app + `npx playwright install chromium`)
```

- **Unit** cover roles/permissions, CSV parsing/mapping, chatbot validation & simulation, automation conditions, broadcast audience/consent, and auth schemas.
 - **Integration** prove tenant isolation under RLS (a non-member cannot read or write another org's data) and the ticket status-history trigger.
 - **E2E** cover protected-route redirect, sign-in/out, and opening a conversation and sending a reply.
-

Troubleshooting

- **`npm not found`** (Windows PATH quirk): run
`node node_modules/next/dist/bin/next dev.`
- **DB host won't resolve:** new Supabase projects use the **connection pooler** host (`aws-0-<region>.pooler.supabase.com`, user `postgres.<ref>`), not `db.<ref>.supabase.co`. Copy the pooler URI from Dashboard → Settings → Database and put it in `SUPABASE_DB_URL`.
- **Realtime shows “Offline”:** confirm migration `0009` was applied (`npm run db:push`), that you're signed in, and that the anon key in `.env.local` matches the project.
- **Seed says a value is missing:** ensure `NEXT_PUBLIC_SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` and `SUPABASE_DB_URL` are all set in `.env.local`.